



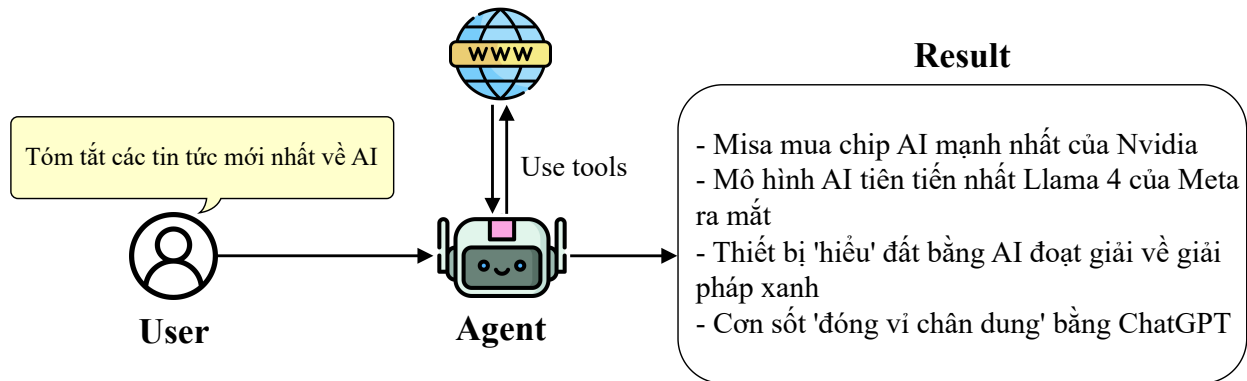
LLM Series: Getting Started with AI Agent Libraries

Dương Đình Thắng

Đinh Quang Vinh

I. Giới thiệu

AI Agents (Tạm dịch: Các tác tử AI) là các hệ thống phần mềm có khả năng tự động nhận yêu cầu từ người dùng, lập kế hoạch và thực hiện chuỗi hành động để hoàn thành mục tiêu, mà không cần người dùng chỉ định từng bước cụ thể. Sự phát triển của các mô hình ngôn ngữ lớn (LLM) với khả năng hiểu và sinh ngôn ngữ tự nhiên đã mở ra một thể hệ tác tử AI mới, có thể kết hợp suy luận (reasoning) và hành động (acting) một cách linh hoạt. Những tác tử này tích hợp LLM cùng các công cụ (tools) bên ngoài để thu thập và xử lý thông tin, cho phép tự động hóa các tác vụ từ đơn giản như tra cứu thông tin đến phức tạp như tổng hợp và phân tích dữ liệu quy mô lớn.



Hình 1: Minh họa bài toán tổng hợp tin tức.

Trong bối cảnh độc giả bận rộn và tin tức cập nhật liên tục, duyệt thủ công vừa tốn thời gian vừa dễ bỏ sót nội dung quan trọng. Do đó, bài viết này hướng đến tự động tổng hợp tin tức, giúp người dùng nắm bắt nhanh các đầu mục nổi bật trong ngày qua AI Agent. Như vậy, bài toán tổng hợp tin tức tự động có Input/Output như sau:

- **Input:** Yêu cầu của người dùng, ví dụ “Tổng hợp tin tức AI hôm nay”.
- **Output:** Danh sách tiêu đề và đường dẫn của các tin tức mới nhất.

Để tiện theo dõi, các bạn có thể tham khảo bảng các thuật ngữ được dùng trong bài tại Mục IV.

Mục lục

I.	Giới thiệu	1
II.	Thư viện smolagents	4
II.1.	Tổng quan	4
II.2.	Kiến trúc	5
II.3.	Quy trình hoạt động	5
II.4.	Các loại Agent trong smolagents	10
II.5.	Tools	13
III.	Cài đặt Agent tổng hợp tin tức	17
III.1.	Cài đặt và import thư viện	17
III.2.	Định nghĩa các công cụ (Tools)	17
III.3.	Khởi tạo Agent	20
III.4.	Chạy agent với một nhiệm vụ cụ thể	21
III.5.	Quá trình thực hiện của Agent	21
III.6.	Đưa Agent lên Hugging Face Hub (tùy chọn)	23
IV.	Tài liệu tham khảo	24
	Bảng thuật ngữ	25
	Phụ lục	27

Hiện tại, có một số thư viện/framework hỗ trợ phát triển AI Agents, cho phép chúng ta triển khai tác tử một cách nhanh chóng. Song, ở trong phạm vi của bài viết này, chúng ta sẽ lựa chọn **smolagents** (Hình 2) làm công cụ minh họa để xây dựng tác tử thực hiện nhiệm vụ tổng hợp tin tức.



Hình 2: Logo của thư viện **smolagents**. Nguồn: [Link](#).

Hệ thống sẽ lưu giữ chi tiết từng bước: yêu cầu đầu vào, truy vấn gửi đến công cụ, kết quả trả về và tổng hợp thành kết quả cuối cùng. Những công đoạn đó, thư viện **smolagents** sẽ hỗ trợ triển khai một tác tử tự động xử lý một cách liên tục. Chỉ với một số cài đặt Python đơn giản, chúng ta hoàn toàn có thể xây dựng một tác tử AI có khả năng:

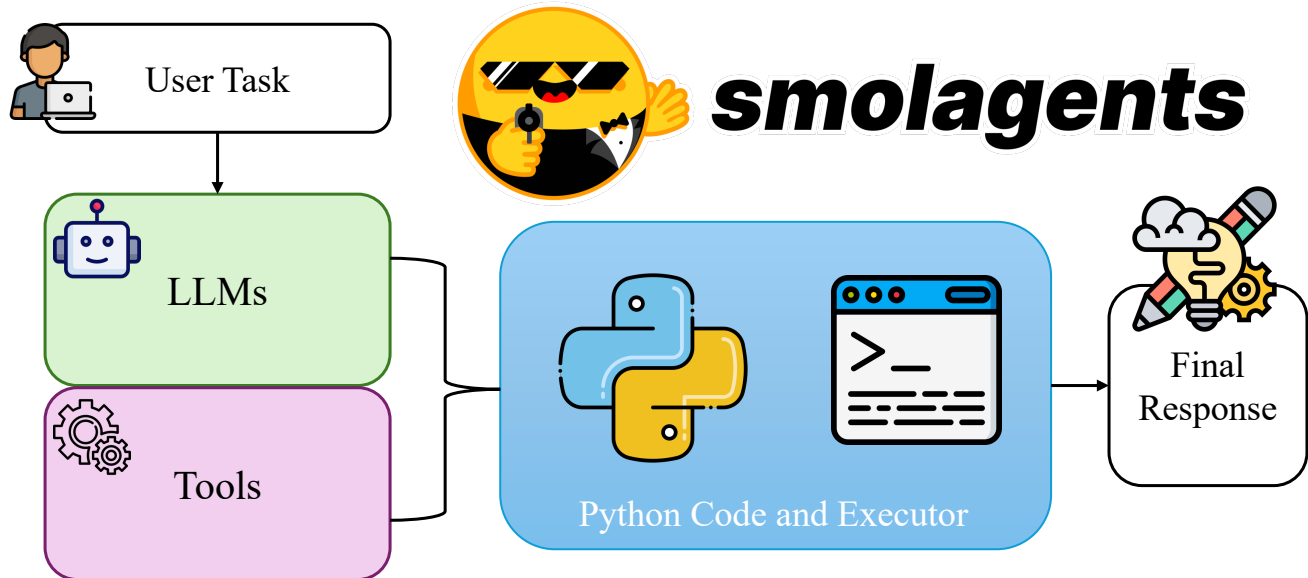
- Tìm kiếm và thu thập tin tức từ nhiều nguồn (web, API).
- Lọc và loại bỏ các mục tin không phù hợp hoặc trùng lặp.
- Đánh giá mức độ liên quan và ưu tiên các bản tin.
- Trích xuất tiêu đề, đường dẫn và tóm tắt ngắn gọn.

II. Thư viện smolagents

II.1. Tổng quan

smolagents (minh họa ở Hình 3) là một thư viện đơn giản nhưng mạnh mẽ được phát triển bởi Hugging Face, giúp xây dựng các AI agents chỉ với vài dòng code. Mục tiêu chính của thư viện là đơn giản hóa quá trình phát triển ứng dụng AI thông minh, giảm thiểu sự phức tạp và các lớp trừu tượng không cần thiết. Cụ thể, thư viện có các đặc điểm nổi bật sau:

- **Tính đơn giản tối đa:** Logic cho các agents trong smolagents chỉ khoảng 1000 dòng code, giúp code dễ đọc, dễ hiểu và dễ debug, giảm thiểu các lớp trừu tượng không cần thiết.
- **Cách tiếp cận Code-first:** smolagents nổi bật với phương pháp tiếp cận “code-first” qua các agent viết các hành động của mình bằng mã Python thay vì JSON hay text blobs. Cách tiếp cận này được chứng minh là hiệu quả hơn đáng kể so với các phương pháp truyền thống.
- **Hỗ trợ đa dạng các mô hình:** Thư viện tương thích với nhiều loại mô hình ngôn ngữ lớn từ các nguồn khác nhau như Hugging Face Hub, OpenAI, Anthropic, LiteLLM...
- **Tích hợp chặt chẽ với hệ sinh thái Hugging Face:** smolagents dễ dàng kết nối với Hugging Face Hub, hỗ trợ chia sẻ tools, sử dụng Gradio Spaces và tích hợp với các mô hình Transformers.



Hình 3: Minh họa các thành phần chính trong thư viện smolagents.

II.2. Kiến trúc

Thư viện `smolagents` xây dựng cơ chế hoạt động của các agent dựa trên framework ReAct [1], một phương pháp giúp agent có thể giải quyết nhiệm vụ một cách linh hoạt thông qua chuỗi hành động và suy luận lặp đi lặp lại.

Mỗi agent trong `smolagents` kế thừa từ lớp `MultiStepAgent`, cho phép xử lý nhiệm vụ theo từng bước. Tại mỗi bước, agent sẽ thực hiện một chu trình gồm suy nghĩ (reason), hành động (act), và quan sát phản hồi từ môi trường. Chu trình này tiếp tục lặp lại cho đến khi nhiệm vụ hoàn thành.

Về cấu trúc, một agent được tạo thành từ ba thành phần chính (Hình 4 minh họa):

- **Mô hình (Model):** Là “bộ não” của agent, chịu trách nhiệm suy luận và đưa ra hành động. Có thể sử dụng các mô hình LLM.
- **Công cụ (Tools):** Mở rộng khả năng của agent bằng cách cung cấp chức năng truy xuất thông tin, thực hiện thao tác cụ thể hoặc tương tác với môi trường.
- **Nhật ký (Logs):** Ghi lại toàn bộ quá trình hoạt động như system prompt, yêu cầu người dùng, hành động đã thực hiện, kết quả từ công cụ và phản hồi từ môi trường ở mỗi bước - giúp agent duy trì ngữ cảnh và theo dõi tiến độ.

`smolagents` hiện hỗ trợ hai loại agent chính:

- **CodeAgent:** Loại agent mặc định, viết các lời gọi công cụ bằng mã Python.
- **ToolCallingAgent:** Loại agent này sử dụng định dạng JSON để gọi công cụ.

II.3. Quy trình hoạt động

- **Khởi tạo:** Agent bắt đầu với việc khởi tạo thông tin đầu vào. System prompt được lưu trong `SystemPromptStep`, trong khi yêu cầu của người dùng và ảnh (nếu có) được ghi vào `TaskStep`.
- **Vòng lặp xử lý:** Sau khi khởi tạo, agent bước vào vòng lặp chính để từng bước giải quyết nhiệm vụ cho đến khi công cụ `final_answer` được gọi:
 1. `agent.write_memory_to_messages()` chuyển đổi các log từ các bước trước thành danh sách tin nhắn dạng chat message giúp mô hình tiếp nhận đầy đủ thông tin.
 2. Danh sách tin nhắn này được gửi tới mô hình (`model`), kết quả phản hồi từ mô hình được phân tích để xác định hành động kế tiếp. Tùy vào loại agent, hành động có thể là một đoạn mã Python (`CodeAgent`) hoặc một blob JSON (`ToolCallingAgent`).
 3. Agent thực thi hành động đó và lưu kết quả vào `ActionStep`.
 4. Cuối mỗi bước, các hàm callback trong `agent.step_callbacks` (nếu có) sẽ được gọi để cập nhật trạng thái, ghi log hoặc xử lý các thao tác phụ trợ khác.

- **Lập kế hoạch (Planning) - Tùy chọn:** Nếu được bật, chế độ này cho phép agent cập nhật kế hoạch dựa trên thông tin mới thu thập ở mỗi bước, và lưu lại trong `PlanningStep`. Điều này giúp agent thích ứng tốt hơn với tiến trình thực hiện nhiệm vụ.

II.3.1. Ví dụ minh họa

Tiếp theo, chúng ta sẽ đi sâu vào một ví dụ cụ thể để minh họa quy trình hoạt động từng bước của một agent trong `smolagents`.

```

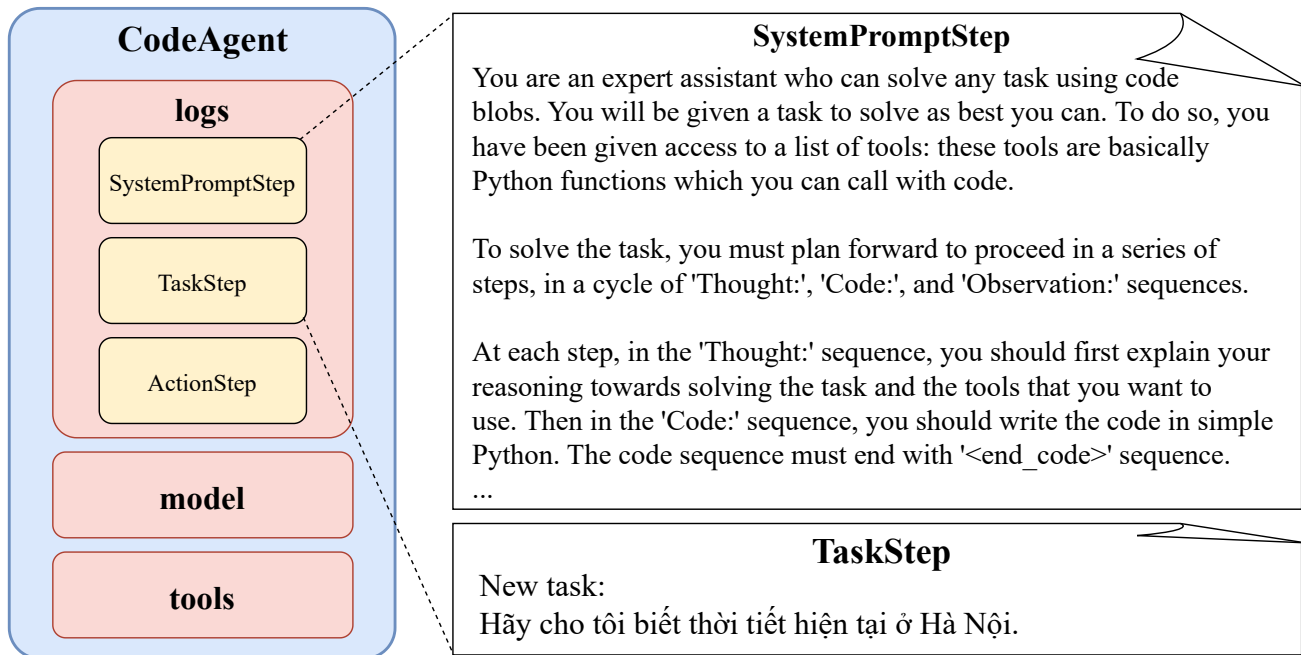
1 from smolagents import CodeAgent, HfApiModel, FinalAnswerTool, tool
2
3 agent = CodeAgent(
4     tools=[get_coordinates, get_current_weather, FinalAnswerTool()],
5     model=HfApiModel()
6 )
7
8 agent.run("Hãy cho tôi biết thời tiết hiện tại ở Hà Nội.")

```

Thông tin về agent:

- **Loại:** `CodeAgent`, hoạt động với mô hình ngôn ngữ lớn.
- **Công cụ sử dụng:**
 - `get_coordinates`: Trả về thông tin kinh độ, vĩ độ của một thành phố.
 - `get_current_weather`: Trả về mô tả thời tiết dựa trên tọa độ.
 - `FinalAnswerTool`: Trả về câu trả lời cho người dùng và hoàn thành nhiệm vụ.
- **Yêu cầu từ người dùng:** “Hãy cho tôi biết thời tiết hiện tại ở Hà Nội.”

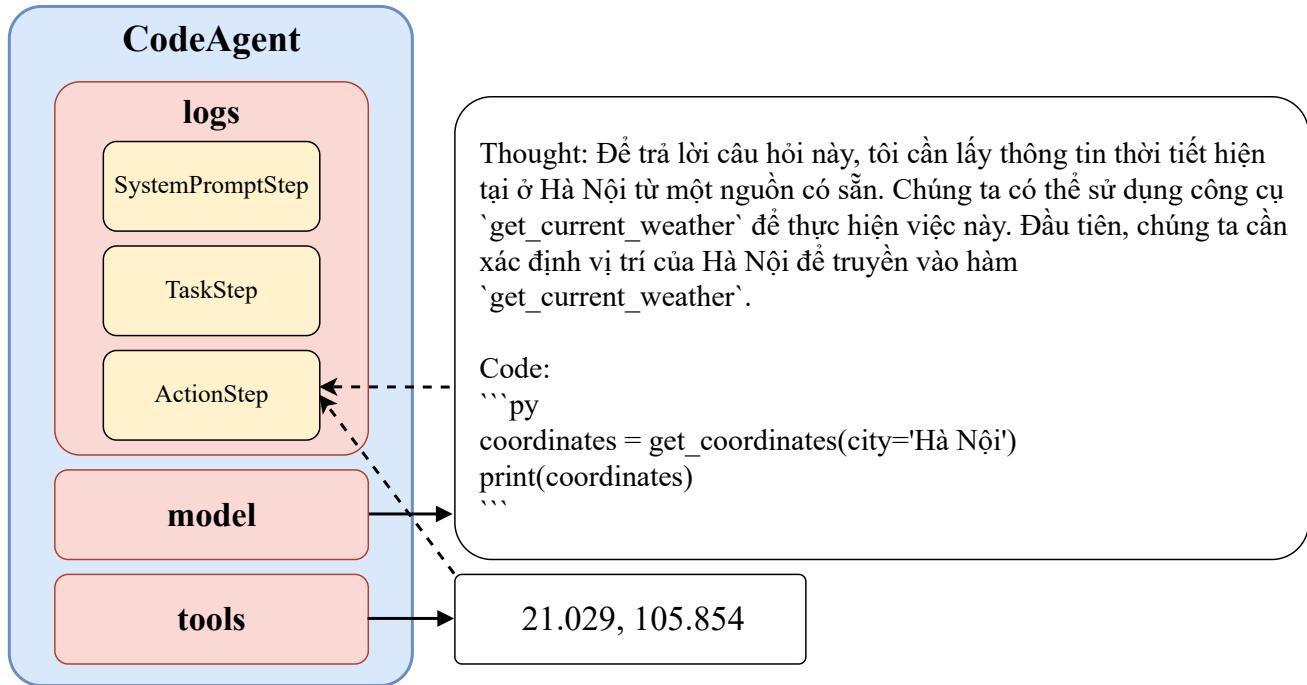
Khởi tạo: Agent được khởi tạo với prompt hệ thống lưu trong `SystemPromptStep`, và yêu cầu từ người dùng được lưu vào `TaskStep`. Quá trình này được minh họa trong Hình 4.



Hình 4: SystemPromptStep và TaskStep của agent. Chi tiết system prompt xem thêm tại [đây](#).

Vòng lặp 1:

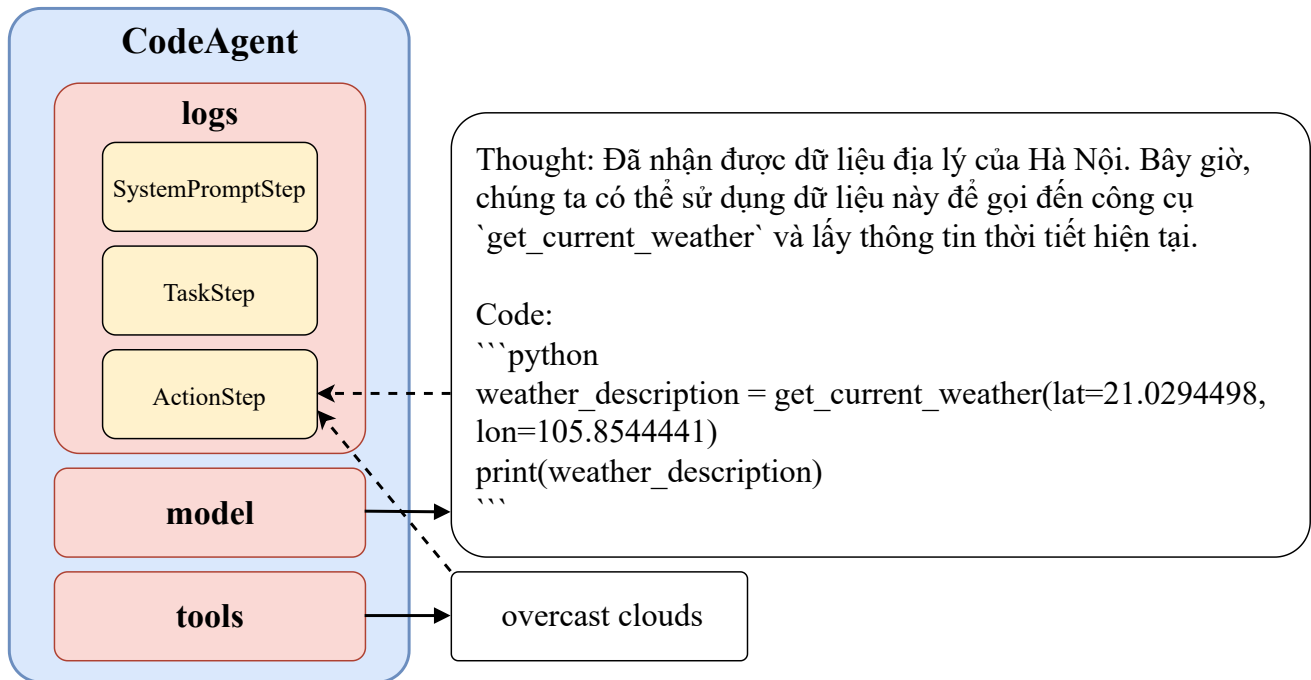
1. Agent chuyển đổi thông tin từ **SystemPromptStep** và **TaskStep** thành các tin nhắn dạng chat message thông qua hàm `agent.write_memory_to_messages()`.
2. Danh sách tin nhắn được gửi tới mô hình (**model**), nhận lại phản hồi gồm suy luận và hành động - cụ thể là một đoạn mã Python gọi công cụ `get_coordinates` với tham số "Hà Nội" (Hình 5).
3. Mã được thực thi, kết quả từ công cụ và mã được lưu vào **ActionStep** và bổ sung vào logs.
4. Không có callback nào được định nghĩa, nên không có hàm nào được gọi ở bước này.



Hình 5: Kết quả trả về của mô hình và công cụ trong vòng lặp đầu tiên.

Vòng lặp 2:

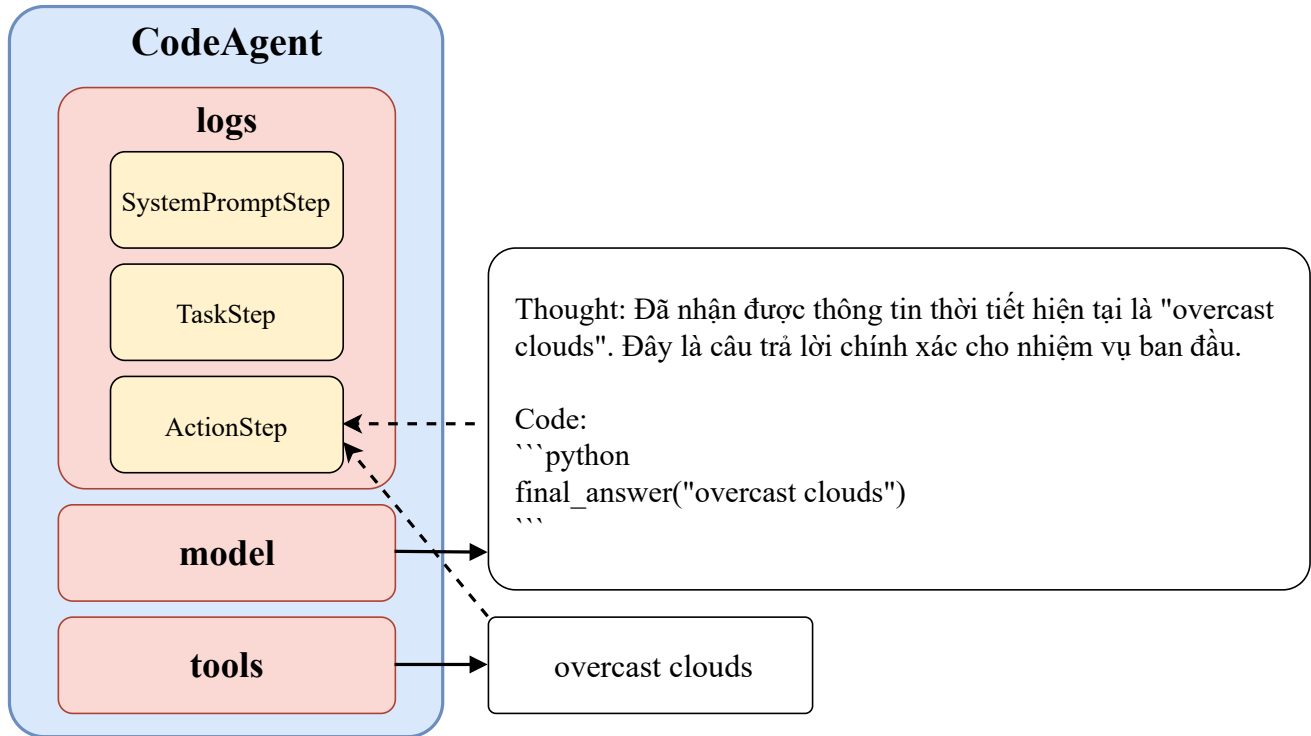
1. Logs từ bước trước được chuyển thành chat message bằng `agent.write_memory_to_messages()`.
2. Mô hình tiếp nhận các tin nhắn này, phản hồi bằng đoạn mã Python gọi công cụ `get_current_weather` với tọa độ vừa thu được (Hình 6).
3. Mã được thực thi, kết quả công cụ và mã được lưu vào **ActionStep** và thêm vào logs.
4. Không có hàm callback được gọi.



Hình 6: Kết quả trả về của mô hình và công cụ trong vòng lặp thứ hai.

Vòng lặp 3:

1. Logs từ hai bước trước được đưa vào `agent.write_memory_to_messages()` để tạo message mới.
2. Mô hình phản hồi bằng đoạn mã gọi công cụ `final_answer` nhằm trả kết quả cuối cùng về cho người dùng (Hình 7).
3. Mã được thực thi, phản hồi cuối được ghi lại và đánh dấu hoàn thành nhiệm vụ.
4. Không có callback nào được gọi ở bước này.



Hình 7: Kết quả trả về của mô hình và công cụ trong vòng lặp cuối.

Kết quả: Sau ba vòng lặp, agent hoàn thành nhiệm vụ và trả lại câu trả lời cuối cùng cho người dùng: *“overcast clouds”*.

II.4. Các loại Agent trong smolagents

II.4.1. CodeAgent

CodeAgent là agent chính trong thư viện **smolagents**, sử dụng mã Python để biểu diễn và thực thi các hành động (*actions*). Khác với cách tiếp cận truyền thống dùng JSON để gọi công cụ, **CodeAgent** sinh trực tiếp mã Python nhằm thực hiện các tác vụ.

Việc sử dụng mã Python thay cho JSON đã chứng minh hiệu quả rõ rệt trong các tác vụ reasoning của agent, theo nhiều nghiên cứu gần đây [2–4]. Lý do là vì ngôn ngữ lập trình như Python vốn được thiết kế để biểu diễn hành động một cách tối ưu, rõ ràng và logic. Một số lợi thế cụ thể khi sử dụng mã nguồn Python:

- **Khả năng tổ hợp cao:** Dễ dàng tái sử dụng, đóng gói các bước thành hàm.
- **Quản lý đối tượng tốt:** Có thể lưu và thao tác với kết quả trung gian như hình ảnh, dữ liệu phức tạp.
- **Tính tổng quát:** Có thể biểu diễn hầu như mọi hành động mà máy tính có thể thực hiện.

- **Tương thích với dữ liệu huấn luyện của LLM:** Các mô hình ngôn ngữ lớn vốn đã được huấn luyện trên lượng lớn mã nguồn, giúp cải thiện hiệu suất trong môi trường reasoning bằng code.

Dưới đây là một ví dụ sử dụng **CodeAgent** để giải bài toán tính số Fibonacci thứ 50:

```
1 from smolagents import CodeAgent, DuckDuckGoSearchTool, HfApiModel
2
3 agent = CodeAgent(
4     tools=[DuckDuckGoSearchTool()],
5     model=HfApiModel()
6 )
7
8 agent.run("Số Fibonacci thứ 50 là bao nhiêu?")
```

Trong ví dụ trên, agent được tạo từ hai thành phần chính:

- Công cụ tìm kiếm DuckDuckGo: Được thêm vào danh sách các tool của agent để hỗ trợ tra cứu thông tin nếu cần.
- Mô hình HfApiModel(Mặc định là mô hình Qwen2.5-Coder-32B-Instruct): Mô hình ngôn ngữ được sử dụng bên trong agent để tư duy và hoàn thành nhiệm vụ.

Khi gọi hàm `run`, agent sẽ phân tích yêu cầu, sinh mã Python tương ứng để giải bài toán, sau đó thực thi và trả về kết quả:

Code được agent sinh ra

```
1 def fibonacci(n):
2     if n <= 0:
3         return 0
4     elif n == 1:
5         return 1
6     else:
7         a, b = 0, 1
8         for _ in range(2, n + 1):
9             a, b = b, a + b
10        return b
11
12 result = fibonacci(50)
13 final_answer(result)
```

Final Answer: Câu trả lời cuối cùng cho nhiệm vụ

12586269025

Như vậy, agent đã tự động sinh ra đoạn code giải bài toán Fibonacci và trả lại kết quả cho người dùng một cách rõ ràng và chính xác.

II.4.2. ToolCallingAgent

ToolCallingAgent là loại agent thứ hai được cung cấp trong thư viện **smolagents**. Khác với **CodeAgent**, agent này sử dụng khả năng gọi công cụ tích hợp sẵn của các mô hình ngôn ngữ lớn để tạo ra các hành động dưới dạng cấu trúc JSON. Đây là phương pháp tiêu chuẩn được sử dụng bởi các hệ thống như OpenAI, Anthropic và nhiều nhà cung cấp khác.

Mặc dù **ToolCallingAgent** vẫn tuân theo quy trình reasoning nhiều bước như **CodeAgent**, điểm khác biệt chính là cách mô tả hành động: thay vì dùng mã thực thi, agent này tạo ra các đối tượng JSON xác định tên tool và tham số tương ứng. Hệ thống sau đó sẽ phân tích và thực thi các công cụ được chỉ định.

Ví dụ sau minh họa cách dùng **ToolCallingAgent** để thực hiện tìm kiếm thông tin thời tiết:

```
1 from smolagents import ToolCallingAgent, DuckDuckGoSearchTool, HfApiModel
2
3 agent = ToolCallingAgent(
4     tools=[DuckDuckGoSearchTool()],
5     model=HfApiModel()
6 )
7
8 agent.run("Thời tiết hiện tại ở Hà Nội như thế nào?")
```

Trong ví dụ trên, agent cũng bao gồm hai thành phần chính:

- **DuckDuckGoSearchTool**: Công cụ tìm kiếm được cung cấp cho agent nhằm hỗ trợ thu thập thông tin từ web.
- **HfApiModel** (mặc định là **Qwen2.5-Coder-32B-Instruct**): Mô hình ngôn ngữ xử lý yêu cầu và sinh ra hành động phù hợp dưới dạng JSON.

Khi gọi hàm **run**, agent sẽ xác định công cụ phù hợp và tạo ra các lệnh gọi công cụ tương ứng. Dưới đây là minh họa quá trình hoạt động gồm hai bước:

Bước 1

Mô hình gọi công cụ

Calling tool: **web_search** with arguments: {query: "thời tiết hiện tại ở Hà Nội"}

Observations: Kết quả trả về từ lời gọi công cụ

0. Thời tiết hiện tại - Hà Nội. (*Source: AccuWeather*)

Thời tiết hiện tại: 11:56. 75°F. Nắng nhẹ. Rất ẩm áp. Dễ chịu. Chỉ số UV tối đa.

1. Thời tiết Hà Nội. (*Source: Trung tâm Dự báo KTTV quốc gia*)

Nhiệt độ: 26°C · Thời tiết: Mây thay đổi, trời nắng · Độ ẩm: 65% · Gió: Đông nam - 1 m/s.

2. Dự báo 3 ngày Hà Nội. (*Source: AccuWeather*)

Thời tiết hiện tại: 21:02. 70°F. Nhiều mây.

...

Bước 2

Mô hình gọi công cụ

Calling tool: `final_answer` with arguments: `{"answer": "Thời tiết hiện tại ở Hà Nội là nắng nhẹ, nhiệt độ khoảng 26°C."}`

Final Answer: Câu trả lời cuối cùng cho nhiệm vụ

Thời tiết hiện tại ở Hà Nội là nắng nhẹ, nhiệt độ khoảng 26°C.

Như vậy, `ToolCallingAgent` đã xác định tool, sinh JSON phù hợp, gọi tool và tổng hợp phản hồi để đưa ra câu trả lời một cách chính xác và rõ ràng.

Mặc dù `smolagents` chủ yếu tập trung vào `CodeAgent` nhờ hiệu suất reasoning tốt hơn nhưng `ToolCallingAgent` vẫn là lựa chọn phù hợp cho các hệ thống đơn giản, không yêu cầu xử lý logic phức tạp.

II.5. Tools

Trong kiến trúc agent, **tools** đóng vai trò là các chức năng bên ngoài mà mô hình LLM có thể gọi đến nhằm mở rộng khả năng hành động - từ tìm kiếm thông tin, xử lý dữ liệu, đến tương tác với hệ thống bên ngoài.

Trong thư viện `smolagents`, tools được thiết kế dưới dạng các hàm Python chuẩn hoá, giúp LLM dễ hiểu, dễ gọi và tương tác một cách chính xác.

II.5.1. Cách mô hình sử dụng tools

Để một agent có thể sử dụng được một tool, nó cần được cung cấp định nghĩa rõ ràng của tool đó, bao gồm các thành phần sau:

- **Tên tool (Name):** Định danh duy nhất của tool, thường là một từ hoặc cụm từ ngắn.

- **Mô tả chức năng (Description):** Diễn giải ngắn gọn mục đích sử dụng của tool.
- **Kiểu và mô tả tham số đầu vào (Inputs):** Gồm tên tham số, kiểu dữ liệu, và vai trò của từng tham số.
- **Kiểu dữ liệu đầu ra (Output):** Cho biết kết quả trả về thuộc kiểu dữ liệu nào.

Ví dụ, một tool tìm kiếm trên web có thể được mô tả như sau:

- **Name:** web_search.
- **Description:** Tìm kiếm thông tin trên internet theo từ khoá.
- **Input:** query (str) - từ khoá cần tìm kiếm.
- **Output:** str - kết quả tìm kiếm dạng văn bản.

II.5.2. Default Toolbox

smolagents cung cấp sẵn một số tool mặc định giúp tăng tốc quá trình phát triển agent. Các tool này được thiết kế để xử lý những nhiệm vụ phổ biến và có thể tích hợp trực tiếp vào hệ thống:

- **PythonInterpreterTool:** Thực thi đoạn mã Python do LLM sinh ra.
- **FinalAnswerTool:** Đánh dấu câu trả lời cuối cùng từ agent.
- **UserInputTool:** Thu thập yêu cầu từ người dùng.
- **DuckDuckGoSearchTool:** Tìm kiếm thông tin web thông qua DuckDuckGo.
- **GoogleSearchTool:** Tìm kiếm thông tin thông qua Google.
- **VisitWebpageTool:** Truy cập và tương tác với một trang web cụ thể.

Bên cạnh các tool mặc định sẵn có, smolagents còn cho phép người dùng dễ dàng mở rộng hệ thống bằng cách tự định nghĩa tools theo nhu cầu.

II.5.3. Tạo Custom Tool

Chúng ta có thể định nghĩa tool mới phù hợp với nhiệm vụ cụ thể theo hai cách: sử dụng decorator `@tool` hoặc kế thừa lớp `Tool` để kiểm soát logic nâng cao.

Cách 1: Dùng Decorator `@tool`

Đây là cách phổ biến và đơn giản nhất để tạo một tool. Chỉ cần viết một hàm Python bình thường, sau đó thêm decorator `@tool` ở đầu hàm. Hệ thống sẽ tự động trích xuất các metadata như tên, kiểu dữ liệu, mô tả từ docstring của hàm để mô hình có thể hiểu và gọi đúng cách.

Lưu ý khi định nghĩa tool:

- Tên hàm nên rõ ràng, phản ánh đúng hành động của tool.
- Dùng `type hints` để định nghĩa kiểu dữ liệu cho các đối số và giá trị trả về.
- Viết docstring rõ ràng, bao gồm phần **Args**: mô tả từng tham số.

Ví dụ: Tool lấy thông tin về ngày hiện tại

```

1 from smolagents import Tool
2
3 @tool
4 def get_todays_date(date_format: str) -> str:
5     """
6     Returns today's date formatted according to the specified pattern.
7
8     Args:
9         date_format (str): The desired date format string.
10                            Examples include "%Y-%m-%d", "%d", "%m", "%Y".
11
12     Returns:
13         str: Today's date as a string formatted with the given pattern.
14     """
15     from datetime import datetime
16     return datetime.now().strftime(date_format)

```

Trong ví dụ trên, tool được định nghĩa với docstring rõ ràng và đầy đủ thông tin về tham số, kiểu dữ liệu và giá trị trả về. Việc import thư viện bên trong hàm đảm bảo rằng khi agent gọi tool, tất cả phụ thuộc sẽ được xử lý đúng lúc, tránh lỗi thiếu import trong quá trình thực thi.

Cách 2: Kế thừa lớp Tool

Với các tool có logic phức tạp hoặc yêu cầu kiểm soát chi tiết (ví dụ: gọi API, xử lý điều kiện đặc biệt...), chúng ta có thể kế thừa lớp `Tool` để định nghĩa hành vi cụ thể và metadata rõ ràng. Một lớp tool tùy chỉnh cần định nghĩa các thành phần sau:

- **name**: Tên tool, được dùng để phân biệt giữa các công cụ.
- **description**: Mô tả ngắn về chức năng, thường được đưa vào prompt hệ thống của agent.
- **inputs**: Từ điển các tham số đầu vào, gồm `type` và `description`.
- **output_type**: Kiểu dữ liệu mà tool trả về.
- **forward**: Phương thức chính chứa logic thực thi của tool.

Ví dụ: Tool gửi lời chào đến người dùng

```

1 from smolagents import Tool
2
3 class GreetTool(Tool):
4     name = "greet_user"
5     description = "Send a friendly greeting to the user using the given name."
6

```

```
7 inputs = {  
8     "name": {  
9         "type": "string",  
10        "description": "The name of the user to greet.",  
11    }  
12 }  
13  
14 output_type = "string"  
15  
16 def forward(self, name: str) -> str:  
17     return f"Hello, {name}! Nice to meet you."
```

Trong ví dụ trên, chúng ta định nghĩa một tool tùy chỉnh bằng cách kế thừa lớp `Tool` và khai báo rõ ràng các thông tin như tên, mô tả, kiểu dữ liệu đầu vào/đầu ra và logic xử lý trong phương thức `forward`.

Cách định nghĩa này giúp LLM hiểu rõ cách sử dụng tool nhờ metadata chi tiết, đồng thời cho phép mở rộng linh hoạt logic xử lý đầu vào/đầu ra theo yêu cầu. Nếu có nhiều tool cần tổ chức và cấu trúc tương tự, cách kế thừa lớp `Tool` sẽ phù hợp hơn so với dùng decorator.

III. Cài đặt Agent tổng hợp tin tức

Trong phần này, chúng ta sử dụng thư viện `smolagents` để xây dựng một AI agent có khả năng thu thập, phân loại và tóm tắt tin tức mới nhất từ trang báo điện tử vnexpress.net. Agent sẽ được trang bị các công cụ tìm kiếm, xử lý và tóm tắt văn bản tiếng Việt, kết hợp với mô hình ngôn ngữ lớn để tự động giải quyết nhiệm vụ từ người dùng.

III.1. Cài đặt và import thư viện

Chúng ta bắt đầu bằng việc cài đặt các thư viện cần thiết:

```
1 !pip install smolagents
2 !pip install "smolagents[transformers]"
```

Sau đó, import các module cần thiết và thiết lập seed để đảm bảo tính tái lập kết quả:

```
1 from smolagents import TransformersModel, CodeAgent, tool, FinalAnswerTool
2 import os
3 import torch
4
5 def set_seed(seed: int):
6     torch.manual_seed(seed)
7     torch.cuda.manual_seed_all(seed)
8     torch.backends.cudnn.deterministic = True
9     torch.backends.cudnn.benchmark = False
10
11 set_seed(42)
```

III.2. Định nghĩa các công cụ (Tools)

Các tool dưới đây giúp mở rộng khả năng của agent, cho phép:

- Truy cập và lấy danh sách các tin tức mới nhất.
- Trích xuất nội dung tin tức chi tiết từ bài viết.
- Tóm tắt nội dung.
- Phân loại xem bài viết có liên quan đến một chủ đề cụ thể không.

1. `fetch_latest_news_titles_and_urls`: Lấy tiêu đề và đường dẫn của các bài viết mới nhất từ trang chủ và các chuyên mục chính của `vnexpress.net`.

```
1 @tool
2 def fetch_latest_news_titles_and_urls(url: str) -> list[tuple[str, str]]:
3     """
4     This tool extracts the titles and URLs of the latest news articles from a news website's
5     homepage.
```

```

5
6  Args:
7      url: The URL of the news website's homepage.
8
9  Returns:
10     list[tuple[str, str]]: A list of titles and URLs of the latest news articles.
11     """
12  import requests
13  from bs4 import BeautifulSoup
14
15  article_urls = []
16  article_titles = []
17  navigation_urls = []
18
19  response = requests.get(url)
20  soup = BeautifulSoup(response.text, "html.parser")
21
22  navigation_bar = soup.find("nav", class_="main-nav")
23  if navigation_bar:
24      for header in navigation_bar.ul.find_all("li")[2:7]:
25          navigation_urls.append(url + header.a["href"])
26      for section_url in navigation_urls:
27          response = requests.get(section_url)
28          section_soup = BeautifulSoup(response.text, "html.parser")
29          for article in section_soup.find_all("article"):
30              title_tag = article.find("h3", class_="title-news")
31              if title_tag:
32                  title = title_tag.text.strip()
33                  article_url = article.find("a")["href"]
34                  article_titles.append(title)
35                  article_urls.append(article_url)
36
37  return list(zip(article_titles, article_urls))

```

Giải thích: Tool này lập qua trang chủ và các chuyên mục tin tức của VnExpress, sau đó sử dụng BeautifulSoup để phân tích HTML, trích xuất và trả về tiêu đề cùng với URL của các bài báo.

2. extract_news_article_content: Trích xuất toàn bộ nội dung văn bản từ một bài báo.

```

1  @tool
2  def extract_news_article_content(url: str) -> str:
3      """
4      This tool extracts the content of a news article from its URL.
5
6      Args:
7          url (str): The URL of the news article.
8
9      Returns:
10         str: The content of the news article.
11         """
12  import requests
13  from bs4 import BeautifulSoup
14
15  response = requests.get(url)

```

```

16 soup = BeautifulSoup(response.text, "html.parser")
17 content = ""
18 for paragraph in soup.find_all("p"):
19     content += paragraph.get_text().strip() + " "
20 return content

```

Giải thích: Tool này truy cập URL đầu vào của bài báo bất kì, tiến hành trích xuất và trả về tất cả nội dung văn bản của bài báo.

3. summarize_news: Tóm tắt nội dung bằng mô hình vit5-base-vietnews-summarization.

```

1 @tool
2 def summarize_news(text: str) -> str:
3     """
4     This tool summarizes the given Vietnamese news text.
5
6     Args:
7         text (str): The Vietnamese news text to be summarized.
8
9     Returns:
10        str: The summarized version of the input text.
11    """
12    from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
13    import torch
14    device = "cuda" if torch.cuda.is_available() else "cpu"
15    model_name = "VietAI/vit5-base-vietnews-summarization"
16    tokenizer = AutoTokenizer.from_pretrained(model_name)
17    model = AutoModelForSeq2SeqLM.from_pretrained(model_name, torch_dtype=torch.bfloat16)
18    model.cuda()
19
20    formatted_text = "vietnews: " + text + " </s>"
21
22    encoding = tokenizer(formatted_text, return_tensors="pt")
23    input_ids = encoding["input_ids"].to(device)
24    attention_masks = encoding["attention_mask"].to(device)
25
26    with torch.no_grad():
27        outputs = model.generate(
28            input_ids=input_ids,
29            attention_mask=attention_masks,
30            max_length=256,
31        )
32
33    summary = tokenizer.decode(
34        outputs[0], skip_special_tokens=True, clean_up_tokenization_spaces=True)
35    return summary

```

Giải thích: Tool sử dụng một mô hình đã fine-tune cho tiếng Việt để tóm tắt nội dung tin tức từ một nội dung văn bản đầu vào.

4. classify_topic: Phân loại một đoạn văn bản có liên quan đến một chủ đề cụ thể hay không bằng zero-shot classification.

```

1 @tool
2 def classify_topic(text: str, topic: str) -> bool:

```

```

3  """
4  This tool classifies whether the given Vietnamese text is related to the specified topic.
5
6  Args:
7      text: The Vietnamese text to be classified.
8      topic: The string representing the topic to be checked.
9
10 Returns:
11     bool: True if the text is related to the topic; False otherwise.
12 """
13 from transformers import pipeline
14 import torch
15 device = "cuda" if torch.cuda.is_available() else "cpu"
16 classifier = pipeline(
17     "zero-shot-classification",
18     model="vicgalle/xlm-roberta-large-xnli-anli",
19     device=device,
20     trust_remote_code=True,
21 )
22
23 candidate_labels = [topic, f"không liên quan {topic}"]
24 result = classifier(text, candidate_labels)
25 predicted_label = result["labels"][0]
26
27 return predicted_label == topic

```

Giải thích: Tool sử dụng mô hình xlm-roberta-large-xnli-anli để phân tích một đoạn văn bản và xác định đoạn văn có liên quan đến chủ đề đầu vào hay không.

III.3. Khởi tạo Agent

Tiếp theo, chúng ta khởi tạo một CodeAgent sử dụng mô hình Qwen2.5-Coder-3B-Instruct từ Hugging Face cùng với các tools đã định nghĩa:

```

1 model = TransformersModel(
2     model_id="Qwen/Qwen2.5-Coder-3B-Instruct",
3     torch_dtype=torch.bfloat16,
4     trust_remote_code=True,
5     max_new_tokens=2000
6 )
7
8 agent = CodeAgent(
9     model=model,
10    tools=[
11        fetch_latest_news_titles_and_urls,
12        summarize_news,
13        extract_news_article_content,
14        classify_topic,
15        FinalAnswerTool()
16    ],
17    additional_authorized_imports=["requests", "bs4"],
18    verbosity_level=2,
19    name="news_agent"

```

20)

Giải thích:

- `tools`: Danh sách các tool mà agent được phép gọi để giải quyết nhiệm vụ.
- `additional_authorized_imports`: Cho phép sử dụng các thư viện bên ngoài như `requests` và `bs4` trong các đoạn mã do mô hình sinh ra.
- `verbosity_level=2`: Hiển thị chi tiết các bước reasoning và hành động của agent, thuận tiện cho việc kiểm tra và debug.

Ta có thể trực quan hóa để quan sát các thành phần của agent đã tạo bằng lệnh:

```
1 agent.visualize()
```

CodeAgent | Qwen/Qwen2.5-Coder-3B-Instruct

✓ Authorized imports: ['requests', 'bs4']
✗ Tools:

Name	Description	Arguments
<code>fetch_latest_news_titles_and_urls</code>	This tool extracts the titles and URLs of the latest news articles from a news website's homepage.	<code>url</code> (`string`): The URL of the news website's homepage.
<code>summarize_news</code>	This tool summarizes the given Vietnamese news text.	<code>text</code> (`string`): The Vietnamese news text to be summarized.
<code>extract_news_article_content</code>	This tool extracts the content of a news article from its URL.	<code>url</code> (`string`): The URL of the news article.
<code>classify_topic</code>	This tool classifies whether the given Vietnamese text is related to the specified topic.	<code>text</code> (`string`): The Vietnamese text to be classified. <code>topic</code> (`string`): The string representing the topic to be checked.
<code>final_answer</code>	Provides a final answer to the given problem.	<code>answer</code> (`any`): The final answer to the problem

Hình 8: Trực quan hoá các thành phần của agent.

III.4. Chạy agent với một nhiệm vụ cụ thể

Chúng ta giao cho agent nhiệm vụ thu thập các tin tức thuộc chủ đề “trí tuệ nhân tạo”:

```
1 task = """
2 Hãy thu thập các tin tức thuộc chủ đề “trí tuệ nhân tạo” trên https://vnexpress.net.
3 """
4
5 result = agent.run(task)
```

III.5. Quá trình thực hiện của Agent

Chi tiết quá trình suy nghĩ của agent được diễn đạt dưới dạng “thought” như sau:

Thought

Để thu thập các tin tức thuộc chủ đề *trí tuệ nhân tạo* trên `vnexpress.net`, tôi sẽ sử dụng công cụ `fetch_lasttest_news_titles_and_urls` để lấy danh sách bài viết. Sau đó, tôi dùng `extract_news_article_content` để lấy nội dung từng bài và `summarize_news` để tóm tắt lại thông tin chính.

Mô hình sẽ sinh ra đoạn mã như sau:

Generated Code

```
1 # Thu thập danh sách bài viết
2 news_titles_and_urls = fetch_lasttest_news_titles_and_urls(url="https://vnexpress.net")
3
4 # Lọc các bài viết có tiêu đề liên quan đến chủ đề
5 selected_titles_and_urls = [
6     (title, url) for title, url in news_titles_and_urls
7     if classify_topic(text=title, topic="trí tuệ nhân tạo")
8 ]
9
10 # Trích xuất nội dung và tóm tắt
11 results = []
12 for title, url in selected_titles_and_urls:
13     content = extract_news_article_content(url=url)
14     summary = summarize_news(text=content)
15     results.append((title, url, summary))
16
17 # In kết quả
18 for title, url, summary in results:
19     print(f>Title: {title}")
20     print(f>URL: {url}")
21     print(f>Summary: {summary}\n")
22
23 # Kết thúc
24 final_answer(results)
```

Và đây là một phần kết quả mà agent trả về:

Final Answer

- Title:** Mô hình AI tiên tiến nhất Llama 4 của Meta ra mắt
URL: <https://vnexpress.net/mo-hinh-ai-tien-tien-nhat-llama-4-cua-meta-ra-mat-4870653.html>
Summary: Meta, công ty mẹ của Facebook, hôm qua ra mắt mô hình ngôn ngữ lớn Llama 4 với phần mềm mô hình trí tuệ nhân tạo (AI).
- Title:** Robot hút bụi đầu tiên trang bị tay cơ học 5 trục
URL: <https://vnexpress.net/robot-hut-bui-dau-tien-trang-bi-tay-co-hoc-5-truc-4870571.html>
Summary: Dòng robot mới có khả năng xử lý khăn, giẻ, hệ thống cấp nước tự động bằng tay cơ học tích hợp AI.
- Title:** Công ty bán dẫn Nhật tìm đường cung cấp chip AI cho Apple, Meta
URL: <https://vnexpress.net/cong-ty-ban-dan-nhat-tim-duong-cung-cap-chip-ai-cho-apple-meta-4870556.html>
Summary: Rapidus, startup bán dẫn Nhật Bản, đang đàm phán với Apple, Google, Amazon để cung cấp chip AI thế hệ mới.

III.6. Đưa Agent lên Hugging Face Hub (tuỳ chọn)

Sau khi xây dựng và thử nghiệm thành công, chúng ta có thể triển khai ứng dụng này thành một web demo trên bất cứ nền tảng host nào để chia sẻ việc sử dụng cho tất cả mọi người. Ở đây, Hugging Face Hub được xem xét sử dụng để triển khai. Trong cài đặt code, việc cập nhật agent sẽ được diễn ra thông qua đoạn code sau:

```
1 agent.push_to_hub("VLAI-AIVN/AI_Agents_smolagents")
```

IV. Tài liệu tham khảo

- [1] S. Yao et al., “React: Synergizing reasoning and acting in language models”, *arXiv preprint arXiv:2210.03629*, 2022.
- [2] X. Wang et al., “Executable code actions elicit better llm agents”, in *Forty-first International Conference on Machine Learning*, 2024.
- [3] Y. Qin et al., “Tool learning with foundation models”, *ACM Computing Surveys*, vol. 57, no. 4, pp. 1–40, 2024.
- [4] K. Yang et al., “If llm is the wizard, then code is the wand: A survey on how code empowers large language models to serve as intelligent agents”, *arXiv preprint arXiv:2401.00812*, 2024.
- [5] H. Face, *Agents course*, <https://github.com/huggingface/agents-course>.
- [6] H. Face, *Smolagents*, <https://github.com/huggingface/smolagents>.

Bảng thuật ngữ

Thuật ngữ	Tạm dịch	Mô tả
<i>AI Agent</i>	Tác tử AI	Hệ thống phần mềm có khả năng tự động nhận yêu cầu từ người dùng, lập kế hoạch và thực hiện chuỗi hành động để hoàn thành mục tiêu mà không cần người dùng chỉ định từng bước cụ thể. Trong bối cảnh LLM, tác tử AI kết hợp suy luận và hành động thông qua việc tích hợp LLM với các công cụ bên ngoài.
<i>Code-first</i>	Phương pháp ưu tiên mã nguồn	Phương pháp thiết kế agent trong đó các hành động được biểu diễn và thực thi trực tiếp dưới dạng mã Python thay vì cấu trúc dữ liệu trung gian như JSON. Phương pháp này tận dụng khả năng sinh mã của LLM và hiệu quả hơn trong các tác vụ reasoning phức tạp.
<i>CodeAgent</i>	Tác tử sinh mã nguồn	Loại agent mặc định trong smolagents , sinh trực tiếp mã Python để biểu diễn và thực thi các hành động. Phương pháp này được chứng minh hiệu quả hơn so với cách tiếp cận dùng JSON trong các tác vụ reasoning phức tạp.
<i>Hugging Face Hub</i>	Nền tảng Hugging Face Hub	Nền tảng chia sẻ mô hình, tập dữ liệu và ứng dụng AI trực tuyến do Hugging Face phát triển. smolagents tích hợp chặt chẽ với nền tảng này, cho phép tải mô hình, chia sẻ tools và triển khai agent dưới dạng web demo.
<i>MultiStepAgent</i>	Tác tử đa bước	Lớp cơ sở của tất cả agent trong smolagents , cho phép xử lý nhiệm vụ theo từng bước thông qua chu trình suy nghĩ, hành động và quan sát phản hồi lặp đi lặp lại cho đến khi công cụ final_answer được gọi.
<i>ReAct</i>	Reasoning and Acting	Framework cho phép agent giải quyết nhiệm vụ thông qua chu trình lặp gồm ba giai đoạn: suy nghĩ (reason), hành động (act) và quan sát phản hồi từ môi trường (observe). Đây là nền tảng kiến trúc của thư viện smolagents .

Thuật ngữ	Tạm dịch	Mô tả
<i>smolagents</i>	Thư viện smolagents	Thư viện mã nguồn mở do Hugging Face phát triển, cho phép xây dựng AI agent chỉ với vài dòng code. Thư viện nổi bật với phương pháp tiếp cận code-first, trong đó agent biểu diễn và thực thi hành động bằng mã Python thay vì cấu trúc JSON.
<i>Tool</i>	Công cụ	Hàm Python được chuẩn hóa mà LLM có thể gọi đến nhằm mở rộng khả năng hành động của agent, từ tìm kiếm thông tin, xử lý dữ liệu đến tương tác với hệ thống bên ngoài. Mỗi tool cần định nghĩa rõ tên, mô tả chức năng, kiểu tham số đầu vào và kiểu dữ liệu đầu ra.
<i>ToolCallingAgent</i>	Tác tử gọi công cụ qua JSON	Loại agent trong smolagents sử dụng khả năng gọi công cụ tích hợp sẵn của LLM, tạo ra các hành động dưới dạng cấu trúc JSON xác định tên công cụ và tham số tương ứng. Phù hợp với các hệ thống đơn giản không yêu cầu xử lý logic phức tạp.
<i>Zero-shot Classification</i>	Phân loại không cần dữ liệu huấn luyện	Kỹ thuật phân loại văn bản mà mô hình có thể xác định nhãn của đầu vào mà không cần được huấn luyện trực tiếp trên tập nhãn đó. Trong bài viết, kỹ thuật này được sử dụng để phân loại bài báo theo chủ đề thông qua mô hình xlm-roberta-large-xnli-anli .

Phụ lục

1. **Resources:** Các file code gợi ý (hint) và bài giải (solution) có thể được tải tại [đây](#). (Lưu ý: Sau khi hết deadline của phần bài này, ad mới copy các nội dung bài giải vào đường dẫn).

2. **Rubric:**

Mục	Kiến Thức	Đánh Giá
II.	- Kiến thức tổng quan về thư viện <code>smolagents</code> và phương pháp tiếp cận code-first. - Kiến trúc agent dựa trên framework ReAct: các thành phần Model, Tools và Logs. - Quy trình hoạt động của agent: khởi tạo, vòng lặp xử lý và chế độ lập kế hoạch tùy chọn. - Phân biệt <code>CodeAgent</code> và <code>ToolCallingAgent</code>. - Cách định nghĩa tools tùy chỉnh bằng decorator <code>@tool</code> và kế thừa lớp <code>Tool</code>.	- Nắm vững kiến trúc và nguyên lý hoạt động của agent trong <code>smolagents</code>. - Phân biệt được sự khác biệt giữa <code>CodeAgent</code> và <code>ToolCallingAgent</code> về cách biểu diễn và thực thi hành động. - Hiểu rõ cách LLM tương tác với tools thông qua metadata (tên, mô tả, kiểu tham số).
III.	- Quy trình xây dựng AI agent tổng hợp tin tức từ <code>vnexpress.net</code>. - Cách định nghĩa và tích hợp các tools tùy chỉnh: thu thập tiêu đề, trích xuất nội dung, tóm tắt và phân loại chủ đề. - Khởi tạo <code>CodeAgent</code> với <code>TransformersModel</code> và các tham số cấu hình liên quan. - Triển khai agent lên Hugging Face Hub.	- Cài đặt và chạy thành công agent với đầy đủ các tools tùy chỉnh. - Xây dựng được pipeline tự động thu thập, phân loại và tóm tắt tin tức theo chủ đề. - Triển khai được ứng dụng agent lên nền tảng Hugging Face Hub dưới dạng web demo.

3. **Q&A:** Bạn có thể đặt thêm câu hỏi về nội dung bài đọc trong group Facebook hỏi đáp tại [đây](#). Tất cả câu hỏi sẽ được trả lời trong vòng tối đa 4 tiếng.

AIO_QAs-Verified

🔒 Nhóm Riêng tư · 1,6K thành viên



Hình 9: Hình ảnh group facebook AIO Q&A.